Bases de données avancées

GLO-4035

Automne 2024

Réalisé par

Zack Dufresne (536 899 030)

Charles-Étienne Dumont (536 960 589)

Charles Brassard (536 783 602)

Christian Fosso (536 904 549)

Rapport final

Travail présenté à

Eloïse Prévôt

8 novembre 2024

Table des matières

Table des matières 2
Introduction 4
Stratégie d'acquisition des données 5
Source et méthode d'extraction 5
Présentation de la ville 5
Source de données 5
Méthode d'obtention des données 6
Analyse des données 6
Présentation d'exemples de données sources 6
Technologies utilisées 6
Langage de programmation 6
Base de données 7
Les détails du processus d'extraction, de transformation et de conversion (ETL) 7
Processus d'acquisition initiale des données 7
Processus d'Acquisition Incrémental des données 7
Processus de transformation des données 8
Schéma du pipeline d'ETL 8
Les détails de la pipeline de données 9
Création d'un parcours 9
Obtention d'un parcours 11
Une explication du plan d'expansion 12
Description 12
Réplication 12
Partitionnement 12
Sécurité 13
Annexes 14
Annexe 1 14
Annexe 2 14
Annexe 3 14

Annexe 4 14
Annexe 5 15
Annexe 6 15
Annexe 7 16
Annexe 8 16
Annexe 9 16
Annexe 10 16
Annexe 11 16
Annexe 12 18
Annexe 13 19
Annexe 14 20
Annexe 15 21
Annexe 16 22
Annexe 17 23
Annexe 18 24
Annexe 19 25
Annexe 20 29

Introduction

L’urbanisation en continuel développement et évolution, les dernières années ont introduit des réseaux de déplacement toujours plus efficace, mais parfois bien plus complexe. C’est sans non plus considérer les particularités topologiques spécifiques à chaque ville qui influe directement sur le développement de ces réseaux. Les restaurants, modèle d’affaire difficile mais très populaire depuis toujours, sont également en constante évolution, particulièrement dans les grandes villes. Plusieurs établissements ouvrent, alors que plusieurs autres mettent la clé sous la porte. C’est deux éléments de notre société apportent bien de variété et de bénéfices dans la vie courante, mais apportent aussi certaines complications dans des cas précis. C’est notamment le cas pour les cyclistes, qui peuvent se retrouver confus face à une quantité grandissante de pistes cyclables. Il peut être compliqué, dans un contexte de promenade à vélo épicurienne, de pouvoir identifier un trajet précis, disponible et sécuritaire permettant également de faire un arrêt à un restaurant convoité. Toujours un problème pour les visiteurs temporaires, cette situation peux maintenant aussi concernée les résidents locaux dépendamment de l’ampleur de la ville. Pour remédier à cette situation, pourquoi ne pas développer une application permettant d’offrir un itinéraire à vélo optimisé qui orientera les utilisateurs dans ce vaste réseau de pistes cyclables et les positionneront par rapport à un restaurants qu’ils souhaitent visiter.

Pour y arriver, il faudra tout d’abord récupérer plusieurs restaurants en activité dans un ville spécifique issue d’une source de données publiques, définie en détail plus loin dans section X. On récupère notamment le nom des restaurants, le type de nourriture servi ainsi que des informations relatives aux horaires d’ouvertures ainsi que leurs localisations.

L’application doit également permettre la récupération des différentes rues de la ville choisie qui possède une piste cyclable pour établir un graphe de toute les rues et coordonnées viables à vélo. Ces données permettent le calcul de la longueur des différentes pistes cyclables, ainsi que le calcul d’un trajet utilisant celles-ci.

En bref, il faut développer une application qui récupère les données qualitatives liées aux restaurants d’une ville précise et qui récupère également les données géographiques de ceux-ci et des pistes cyclables viables de la même ville. Ces données pourront ensuite être utilisé conjointement pour établir un itinéraire à vélo vers des restaurants. Il sera possible d’établir un point de départ, le nombre de résultats désirés ainsi que des types de restaurants souhaités, puis l’application s’occupera de faire les calculs nécessaires à déterminer les différents itinéraires de pistes cyclables viables pour atteindre les restaurants trouvés dans un rayon de 500 mètres entourant le point de départ.

Le public cible pour cette application se définit selon deux différentes dimensions. Elle concerne d'abord, d'un point de vue utilisateurs finaux, des démographiques comme les cyclistes ou les touristes qui souhaitent découvrir une ville à vélo. D'un point de vue marché, cette application concerne les villes possédants des zones urbaines de taille considérable permettant les déplacements à vélo. Le rapport suivant concerne principalement les villes favorisant les cyclistes, on peut notamment penser à différents pays d'Europe ou les déplacements de la sorte sont grandement simplifiés, comme la France par exemple. Le public cible est bien évidemment plus large étant donné la nature de l'application, mais son utilisation serait définitivement plus efficace pour des endroits répondants à ces conditions.

Dans le rapport suivant, nous élaborerons sur la ville choisie pour notre prototype, la source de données utilisée pour récupérer les restaurants, celle utilisée pour les pistes cyclables ainsi qu'une analyse de ces données. Par la suite, nous présenterons les différentes technologies utilisées, que ce soit le langage de programmation ou les types de bases de données utilisés. Nous discuterons ensuite en détail sur le processus ETL (extract, transform, load) que nous avons employés dans le projet, avant de finalement élaborer un scénario d'expansion réaliste pour l'application.

Stratégie d'acquisition des données

Source et méthode d'extraction

Présentation de la ville

La ville que nous avons choisie pour réaliser ce travail est Paris. Celle-ci offre un vaste réseau de pistes cyclables bien développées et une multitude de restaurants allant des bistrots traditionnels aux établissements étoilés. C'est la ville idéale pour une balade à vélo épicurienne.

Source de données

Les données géospatiales utilisées pour ce projet seront récupérées de OpenStreetMap (OSM). C'est une base de données open source qui fournit beaucoup d'informations détaillées sur les routes et points d'intérêts à partir d'une localisation. Grâce au langage de programmation utilisé qui sera présenté plus bas, nous pouvons utiliser la bibliothèque "osmnx" pour récupérer les coordonnées des restaurants de la ville. Nous allons par la suite récupérer à partir de OpenRouteService les itinéraires optimaux à vélo entre deux coordonnées géospatiales, qui combiné avec OSM nous permettra d'obtenir toutes les données nécessaires.

Méthode d’obtention des données

Pour obtenir les données sur les restaurants, nous utiliserons du code similaire à celui présenté à l’Annexe 1, en appliquant un champ “amenity” à restaurant et en ajustant la ville à Rimouski. Pour obtenir les données des itinéraires de pistes cyclables de la ville, nous utiliserons une requête vers l’API de OpenRouteService similaire à celle présentée en Annexe 2.

Analyse des données

Les données sur les restaurants récupérées seront sous la forme d’un GeoDataFrame à partir duquel il est très simple d’extraire les données géospatiales requises pour la suite ou même d’effectuer des opérations de calcul sur ces données. Pour les itinéraires à vélo, la requête présentée en Annexe 2 (où la clé API est omise intentionnellement) retourne un objet JSON qui contient toutes les informations d’itinéraires nécessaires.

Présentation d’exemples de données sources

Voir annexes 3 à 11 pour des exemples de données

Technologies utilisées

Langage de programmation

Pour notre travail longitudinal nous avons décidé d’utiliser Python comme langage de programmation. Ce choix a été adopté notamment pour la simplicité et l'accessibilité de ce dernier qui contribueront à la maintenabilité du système. En ce sens, nous considérons que Python est un outil populaire qui possède une vaste communauté et une documentation abondante ce qui permettra à tous une meilleure autonomie. Par ailleurs, celui-ci permet d’obtenir du code lisible, clair et intuitif ce qui facilitera la compréhension et la communication au sein de l’équipe. De plus, Python peut aussi contribuer à la fiabilité du système. D’une part, il offre un moyen robuste de gérer les exceptions grâce à des blocs try-except ce qui permettra un meilleur contrôle des erreurs. D’une autre part, il permet de détecter les problèmes rapidement et de les corriger au fur et à mesure en effectuant des tests automatisés grâce au framework pytest.

À noter que le langage JavaScript sera utilisé pour implémenter une partie de notre API. Celui-ci permet notamment une bonne réutilisation du code ce qui contribuera à la maintenabilité du système. Pour en ajouter, il permet une bonne fiabilité pour les mêmes raisons que Python et par sa gestion des erreurs avec “promises”.

Base de données

Les types de bases de données que nous avons choisis pour effectuer notre travail longitudinal sont les bases de données orientées document et graphe. Tout d'abord, nous avons choisi le type document car il permet efficacement de stocker les données dans un format qui peut facilement être modifié et qui n'a pas à être constant si les données ne le sont pas elles-mêmes. Ce type est donc bien extensible, ce qui sera un avantage si on doit ajouter de nouvelles données, dont certaines ne seront peut-être pas dans le même format que celles présentes dans la base de données. Dans notre cas, ce type sera utilisé pour stocker les informations sur les restaurants. Nous avons également utilisé le type graphe, car celui-ci permet efficacement de représenter les relations *many-to-many* comme dans notre cas, les réseaux de pistes cyclables. Similairement aux bases de données de types document, celles de types graphes sont extensibles, ce qui sera utile si on doit ajouter de nouveaux types de relations ou étendre les réseaux de pistes cyclables.

Les détails du processus d'extraction, de transformation et de conversion (ETL)

Processus d'acquisition initiale des données

L'extraction initiale des données a été réalisé de deux manières. Premièrement, nous avons utilisé la base de données géographiques OpenStreetMap (OSM) pour extraire les points d'intérêt correspondant aux restaurants. OSM propose une grande variété d'informations, mais nous avons tamisé ceux-ci en garadant uniquement osmid, name, cuisine, coordinates, addr:city, addr:housenumber, addr:postcode, addr:street, opening_hours, phone et description. Ces dernières suffisent pour effectuer une analyse détaillée qui nous permettra de localiser les restaurants. Deuxièmement, nous avons utilisé la même source pour extraire les données relatives aux pistes cyclables. Pour ce cas de figure, nous avons conservé les coordonnées sous forme de liste de points pour une représentation plus exhaustive des chemins.

Voir l'annexe 12 pour le diagramme UML plus détaillé du processus d'extraction

Processus d'Acquisition Incrémental des données

Dans le cadre de ce projet, un processus d'acquisition incrémental des données a été défini afin de maintenir les bases de données MongoDB et Neo4J à jour. Cela permet d'éviter les doublons et de gérer efficacement les mises à jour et ajouts

futurs. Dans le cas de MongoDB, nous vérifions pour chaque restaurant s'il existe déjà dans la base de données en s'appuyant sur l'ID de ce dernier. Si une correspondance est trouvée, nous mettons à jour les informations nécessaires. Dans le cas contraire, un nouveau document est ajouté à la collection. Dans le cas de Neo4J, pour chaque segment de piste cyclable, nous utilisons des requêtes *MERGE* afin de vérifier l'existence des nœuds de début et de fin, ainsi que de la relation entre eux. Si le segment est nouveau, il est ajouté en tant que nouvelle relation entre les nœuds correspondants.

Voir l'annexe 13 pour le diagramme illustrant le flux des étapes dans le processus d'acquisition incrémental pour MongoDB et Neo4j.

Processus de transformation des données

Pour obtenir des données plus compactes et parlantes, nous avons réaliser quelques transformations simples. Pour commencer, nous avons standardiser les adresses des restaurants en appliquant un format canadien. Ainsi, chaque adresse est désormais représentée sous la forme "Numéro de maison, Rue, Ville, Code postal". Par la suite, nous avons traduit les colonnes en français et renommer certaines d'entre-elles pour qu'elles soient plus adapté à notre contexte. Effectivement, la colonne *name* a été traduite par *nom*, et la colonne cuisine a été remplacée par *type_de_restaurant*. Finalement, nous avons procédé à la classification des restaurants et services. En ce sens, les différents restaurants sont maintenant divisés par type selon la colonne *type_de_restaurant* et on a ajouté la colonne *type_de_service* pour classifier les services. Cette dernière a été créé à partir de la description du restaurant. Par exemple, "emporté" est classé comme "Service à emporter", "uber eats" comme "Service de livraison", "buffet" ou "À volonté" comme "Buffet / À volonté". Cette distinction est essentielle pour mieux comprendre l'expérience de restauration proposée.

Schéma du pipeline d'ETL

Les deux types de base de données utilisés dans le cadre de notre application sont bien complémentaires. En effet, nous utilisons MongoDB comme stockage principal de nos restaurants, où le format JSON offert par celui-ci nous permet d'obtenir des données structurées et flexibles pour chacun des restaurants. Neo4j nous permet ensuite d'établir des relations avec les pistes cyclables et les restaurants.

Ainsi, les données transformées des restaurants sont chargées dans MongoDB pour permettre des recherches et des analyses rapides sur les options de restauration. Nous commençons évidemment par une connexion à notre base de données. Par la suite, nous éliminons les doublons avant de convertir le fichier

CSV contenant les données de restaurants transformées en liste de dictionnaires Python. Cette dernière étape est prérequis à l'insertion des données dans notre collection MongoDB.

Par ailleurs, les pistes cyclables sont structurées dans Neo4j en tant que réseau de connexions entre différents points géographiques, ce qui est favorable pour des requêtes orientées graphes, comme la recherche de trajets. Après avoir établis la connexion à notre base de données Neo4j, nous procédons à l'élimination des doublons puis au chargement des données. Par la suite, nous utilisons le fichier CSV contenant les données sur les pistes cyclables et des commandes *MERGE* pour créer des nœuds basés sur les coordonnées géographiques et leurs relations. Finalement, nous utilisons aussi le même procédé avec le fichier contenant les données des restaurants transformés pour la création des nœuds.

Voir l'annexe 4 pour un schéma du pipeline d'ETL global.

Les détails de la pipeline de données

Création d'un parcours

La création d'un parcours consiste à générer un itinéraire cyclable à partir d'un point de départ donné. Cet itinéraire inclut des arrêts dans des restaurants correspondant à des critères précis tels que le type de restaurant, la distance totale du trajet (±10%), et le nombre d'arrêts souhaités.

Le processus global de création d'un parcours se décompose en plusieurs étapes :

1. **Identifier le nœud cyclable le plus proche** du point de départ dans un rayon de 500 mètres.
2. **Parcourir le réseau cyclable** à partir de ce nœud en respectant les critères de distance totale et de nombre d'arrêts dans des restaurants connectés au réseau.
3. **Sélectionner des restaurants** connectés aux nœuds cyclables en évitant les doublons et en priorisant ceux les plus proches.
4. **Assembler un GeoJSON** contenant les points des restaurants et les segments cyclables (MultiLineString).

Le diagramme de flux en Annexe X présente une représentation visuelle du parcours.

L’algorithme en jeux pour permettre la création d’un parcours selon les requêtes effectués comporte plusieurs étapes. La séquence des calculs généraux peut être résumé par ces points :

1. **Recherche initiale :** Identifier le nœud cyclable le plus proche (à l’aide de point.distance dans Neo4j).
2. **Parcours des segments :** Utiliser les relations CYCLEWAY pour naviguer dans le réseau cyclable tout en cumulant les distances.
3. **Vérification des restaurants :** Rechercher les relations CONNECTED_TO pour trouver des restaurants à proximité qui respecte les distances possibles pour valider la longueur du trajet désiré.
4. **Construction du GeoJSON :** Ajouter chaque restaurant et chaque segment au résultat sous forme de Points et de MultiLineString.

L’algorithme utilisé, présenté plus en détail, se définit par le pseudocode suivant :

Pseudocode de l’algorithme :

Entrées : starting_point, length, number_of_stops, types

Sortie : GeoJSON du parcours

1. Trouver le nœud cyclable le plus proche (distance ≤ 500 m).
2. Initialiser total_distance = 0, stops_added = 0, current_node = starting_point.
3. Tant que total_distance < length * 1.1 et stops_added < number_of_stops :
   a. Trouver les restaurants connectés au nœud courant.
   b. Si un restaurant valide est trouvé :
      - Ajouter le restaurant au GeoJSON.
      - Mettre à jour current_node et total_distance.
      - Incrémenter stops_added.
   c. Sinon :
      - Naviguer vers le prochain nœud cyclable connecté.
4. Retourner le GeoJSON si les critères sont remplis.

Finalement, on devrait aussi mentionner que les restaurants sont sélectionnés selon les règles suivantes :

1. **Filtrage par type :** Seuls les types précisés dans la requête utilisateur sont pris en compte (si spécifiés).
2. **Priorité à la proximité :** Les restaurants les plus proches (à < 100 mètres du nœud) sont priorisés.
3. **Aucun doublon :** Les restaurants déjà visités sont exclus.

Obtention d'un parcours

La procédure utilisé pour obtenir un parcours viable possède deux éléments distincts mais qui agissent conjointement à l'obtention d'un bon résultat. Le premier élément correspond à l'établissement d'une requête utilisateur par http REST comme ceci :

Requête utilisateur

1. L'utilisateur envoie une requête POST avec les paramètres suivants :
    - startingPoint : Coordonnées (latitude, longitude).
    - length : Longueur totale du parcours.
    - numberOfStops : Nombre d'arrêts souhaités.
    - Type : Liste des types de restaurants (optionnel).

Le second élément de la procédure d'obtention d'un parcours correspond au traitement réalisé par le serveur à la suite de la requête utilisateur décrite ci-dessus. Ce traitement se définie comme ceci :

Traitement serveur

1. Le serveur vérifie les données entrées (validité des coordonnées, longueur minimale, etc.).
2. Le serveur gère les calculs pour générer un parcours valide en fonction des critères.
3. Le serveur renvoie un GeoJSON comprenant :
    - Les Points représentant les restaurants visités.
    - Les MultiLineString représentant les segments cyclables parcourus.

Le diagramme de séquence circulaire à l'Annexe X offre une représentation visuelle de ce processus.

Une explication du plan d'expansion

Description

Notre système permet actuellement à des utilisateurs d'obtenir des parcours à vélo sur lesquels sont situées des restaurants dans la ville de Paris. Toutefois, il pourrait être intéressant d'étendre la zone de recherche sur d'autres régions européennes voisines tel que la Normandie. Évidemment cela aura un impact sur le taux de requêtes par seconde (RPS) puisque notre application pourra intéresser un plus grand nombre d'utilisateur. Effectivement, cette région compte à elle seule environ 3 327 000 millions d'habitants [1] et « En 2023, ce ne sont pas moins de 2,2 millions de sorties cyclistes qui ont été enregistrées sur les principaux itinéraires de la région » [2]. Par conséquent, on estime une hausse d'environ 50 RPS notamment les fins de semaines. Par ailleurs cela pourrait aussi avoir un impact sur le volume de données à emmagasiner. En ce sens, la région de Normandie possède une quantité importante de pistes cyclables : « Une fréquentation qui reflète l'attractivité des 1 700 km de véloroutes que compte la région. »[3] et de restaurants, on parle de plus de 470 restaurants selon la carte-plan de Via Michelin pour la région de Normandie[4] . Ainsi, nous prévoyons un dédoublement des données de notre application autant pour les restaurants de MongoDB que pour les pistes cyclables de Neo4J.

Réplication

Afin d'améliorer l'extensibilité et la fiabilité de notre application, nous utiliserons une réplication maître-esclave. Ainsi, notre système sera composé d'un nœud primaire (le maître) et de nœuds secondaires (les esclaves) qui répliquerons le nœud primaire. Par conséquent, cela pourrait réduire considérablement le taux RPS en distribuant les lectures sur plusieurs instances de notre base de données et, par le fait même, la latence de notre application.

Partitionnement

Nous fragmenterons nos données en sous-ensembles pour faciliter l'extensibilité et améliorer la performance de notre application. Pour les restaurants, nous

---

[1] Wikipedia, « Normandie », dernière modification le 30 novembre 2024, consulté le [30 novembre 2024], https://fr.wikipedia.org/wiki/Normandie

[2] Weelz, « Le tourisme à vélo, un levier économique pour les territoires – Exemple avec la Normandie », Ouset-France, consulté le [30 novembre 2024], https://weelz.ouest-france.fr/le-tourisme-a-velo-un-levier-economique-pour-les-territoires-exemple-avec-la-normandie/

[3] Idem.

[4] ViaMichelin, « Restaurants en Normandie », consulté le [30 novembre 2024], https://www.viamichelin.fr/cartes-plans/restaurants/france/normandie

implémenterons un partitionnement horizontal basé sur la localisation géographique des données. En ce sens, chaque région (Paris, Normandie) correspondra à un fragment différent. Pour ce qui est des pistes cyclables, on procédera sensiblement de la même manière. Effectivement, nous formerons des sous-graphes pour chaque région qui seront stockés dans des nœuds différents. Toutefois, il faudra utiliser des nœuds-pivots permettant de connecter les différentes sous-graphes lorsqu'un parcours partage plusieurs régions. Nous estimons que cette méthode de partitionnement permettra d'accroître le taux de requêtes par seconde car les différents nœuds seront responsables d'un ensemble de données restreint. Par exemple, les utilisateurs recherchant des parcours en Normandie n'interrogeront que les fragments contenant des données propres à cette région ce qui réduira la latence globale.

Sécurité

Un utilisateur pourra se connecter à notre application pour rechercher un itinéraire à vélo permettant des arrêts dans divers restaurants. L'application devra alors interroger la base de données MongoDB pour obtenir des informations sur les restaurants et Neo4J pour calculer les trajets via les pistes cyclables. Afin d'éviter que l'utilisateur puisse exploiter une faille pour injecter des requêtes malveillantes et corrompre les données, il pourrait être judicieux d'implémenter une validation stricte des entrées utilisateurs. Par exemple, nous pourrions vérifier le format des coordonnées géographiques ou encore spécifier les mots-clés autorisés. Dans le cas de Neo4J, nous pourrions utiliser des paramètres dans les requêtes *Cypher*.

Un administrateur pourra ajouter de nouveaux restaurants ou de nouvelles pistes cyclables à nos bases de données. Celui-ci pourra insérer de nouveaux documents dans MongoDB et créer de nouveaux nœuds et/ou relations dans Neo4J. Afin d'éviter qu'un individu mal intentionné puisse exploiter les identifiants d'un administrateur pour manipuler ou supprimer des données critiques, il pourrait être intéressant de mettre en place un contrôle d'accès basé sur les rôles. Par exemple, nous pourrions restreindre l'accès en écriture aux différents documents de notre base de données MongoDB. Par ailleurs, l'erreur étant humaine, un administrateur pourrait ajouter des données incorrectes ou supprimer des données importantes par inattention. Afin de limiter les conséquences de telles erreurs, il pourrait être judicieux encore une fois de limiter les permissions de l'administrateur par l'utilisation des rôles dans le cas de MongoDB et en mettant en place des sauvegardes automatiques des bases de données à intervalles réguliers ce qui nous permettra de restaurer le système en cas de problème majeur.

Annexes

Annexe 1

Exemple de code Python pour récupérer des “amenity” restaurant de la ville Rimouski

```python
import osmnx as ox

tags = { "amenity": "restaurant", "highway": "cycleway" }

gdf = ox.features_from_place("Rimouski, Quebec, Canada", tags)
```

Annexe 2

Exemple de requête envoyé vers l’API de OpenRouteService pour l’itinéraire en vélo entre deux points (dans notre cas, les points seront des coordonnées de restaurants, la clé est omise par standard de confidentialité)

```
GET https://api.openrouteservice.org/v2/directions/cycling-regular?api_key=api-key&start=8.681495,49.41461&end=8.687872,49.420318
```

Annexe 3

Exemple de structure d’un GeoDataFrame créé par le code de l’Annexe 1 qui contiendra les restaurants de la ville (généré par intelligence artificielle ChatGPT 4o)

	osmid	name	geometry	amenity	addr:street
0	123456789	Chez Gourmet	POINT (-68.52 48.44)	restaurant	Rue Principale
1	987654321	Bistro du Port	POINT (-68.53 48.45)	restaurant	Avenue de la Mer

Annexe 4

Exemple d’objet JSON retourné par une requête de format similaire à celle présentée à l’Annexe 2 (les champs distance (en mètres) et duration (en secondes) correspondent aux détails du trajet global, le champ concaténé “steps” énumère les différentes étapes du trajet avec instruction de déplacement progressive)

```
{
    "type": "FeatureCollection",
    "bbox": [8.681423, 49.414599, 8.690123, 49.420514],
    "features": [
      {
        "bbox": [8.681423, 49.414599, 8.690123, 49.420514],
        "type": "Feature",
        "properties": {
          "segments": [
            {
              "distance": 1408.8,
              "duration": 281.9,
              "steps": [ ···
              ]
            }
          ],
          "way_points": [0, 23],
          "summary": {
            "distance": 1408.8,
            "duration": 281.9
          }
        },
```


Annexe 5

Exemple d'erreur de collecte de données, la ville de Los Angeles n'étant pas au Québec

```
import osmnx as ox

tags = { "amenity": "restaurant", "highway": "cycleway" }

gdf = ox.features_from_place("Los Angeles, Quebec, Canada", tags)
```

Annexe 6

Exemple d'erreur de collecte de données en omettant l'emplacement désiré

```
import osmnx as ox

tags = { "amenity": "restaurant", "highway": "cycleway" }

gdf = ox.features_from_place(tags)
```

Annexe 7

Exemple d'erreur de collecte de données, récupérant les banques au lieu des restaurants, en plus d'une faute dans le nom de la ville

```
import osmnx as ox

tags = { "amenity": "bank", "highway": "cycleway" }

gdf = ox.features_from_place("Rimousky, Quebec, Canada", tags)
```

Annexe 8

Exemple d'erreur d'extraction de données, le mode de transport est la voiture, ce qui ne correspond pas à ce que l'on souhaite

```
GET https://api.openrouteservice.org/v2/directions/driving-car?api_key=your-api-key&start=
8.681495,49.41461&end=8.687872,49.420318
```

Annexe 9

Exemple d'erreur d'extraction de données, la clé API est omise et laissé à nulle, ne permettant pas la récupération des données

```
GET https://api.openrouteservice.org/v2/directions/cycling-regular?api_key=null&start=
8.681495,49.41461&end=8.687872,49.420318
```

Annexe 10

Exemple d'erreur de données, utilisant des coordonnées géospatiales invalides en dehors des limites conventionnellement établies

```
GET https://api.openrouteservice.org/v2/directions/cycling-regular?api_key=api-key&start=
95.000000,-195.000000&end=8.687872,49.420318
```

Annexe 11

Exemple d'erreur de formatage de données, utilisant de virgules "," pour les coordonnées de départ à la place de points "."

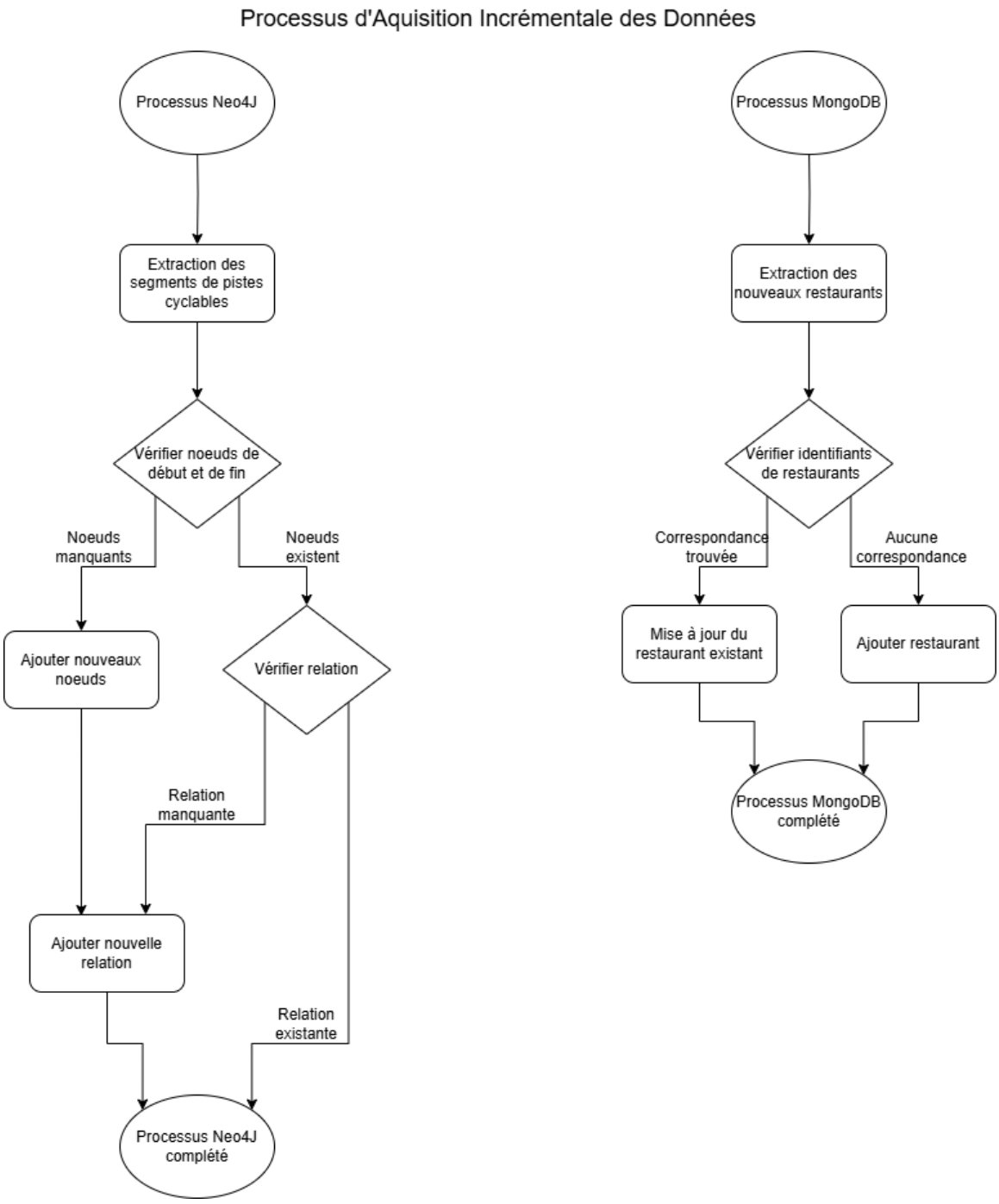
GET https://api.openrouteservice.org/v2/directions/cycling-regular?api_key=api-key&start=8,681495,49,41461&end=8.687872,49.420318

Annexe 12

Diagramme UML du processus d’extraction

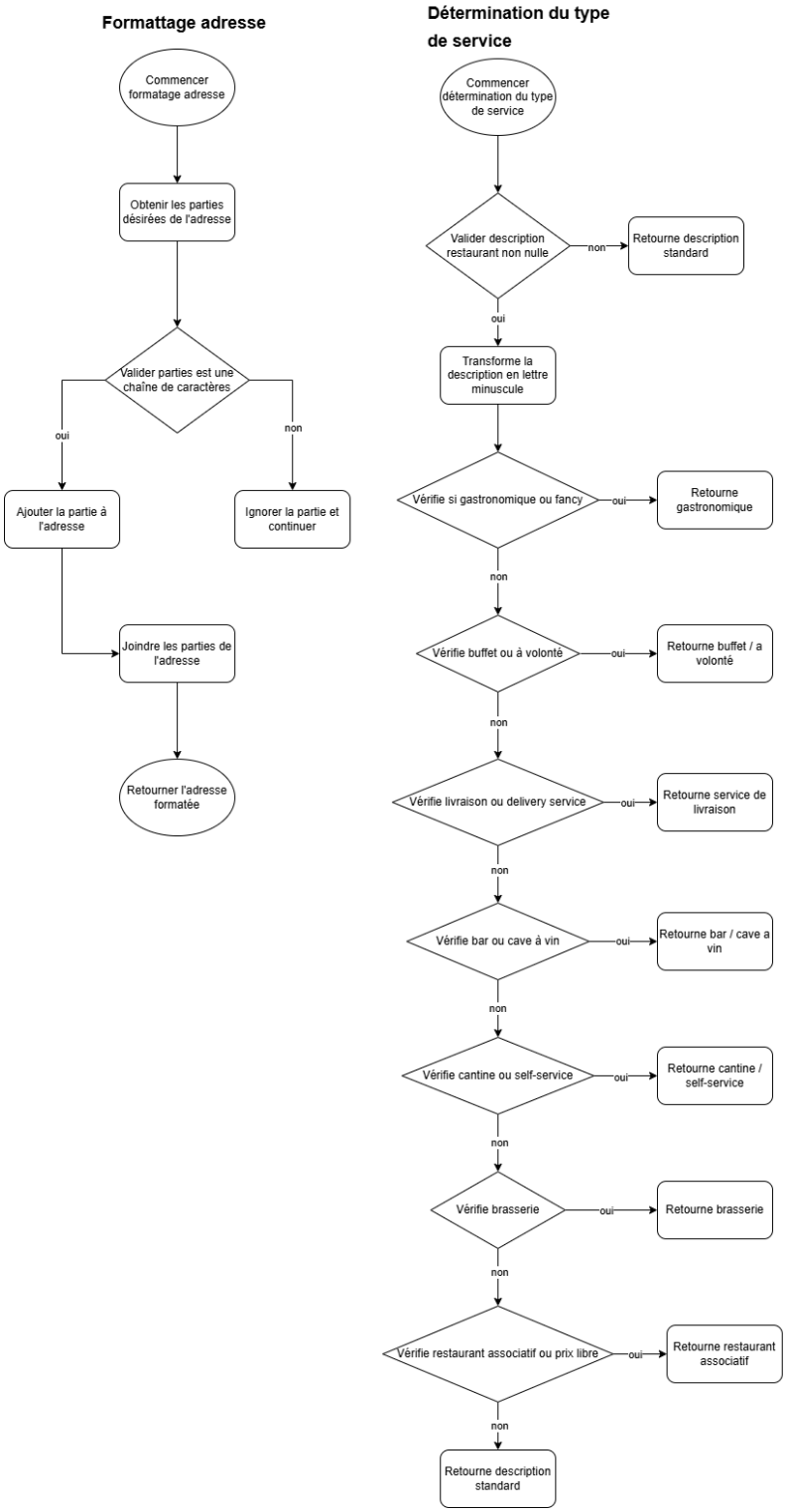
Annexe 13

Processus d'acquisition incrémentale



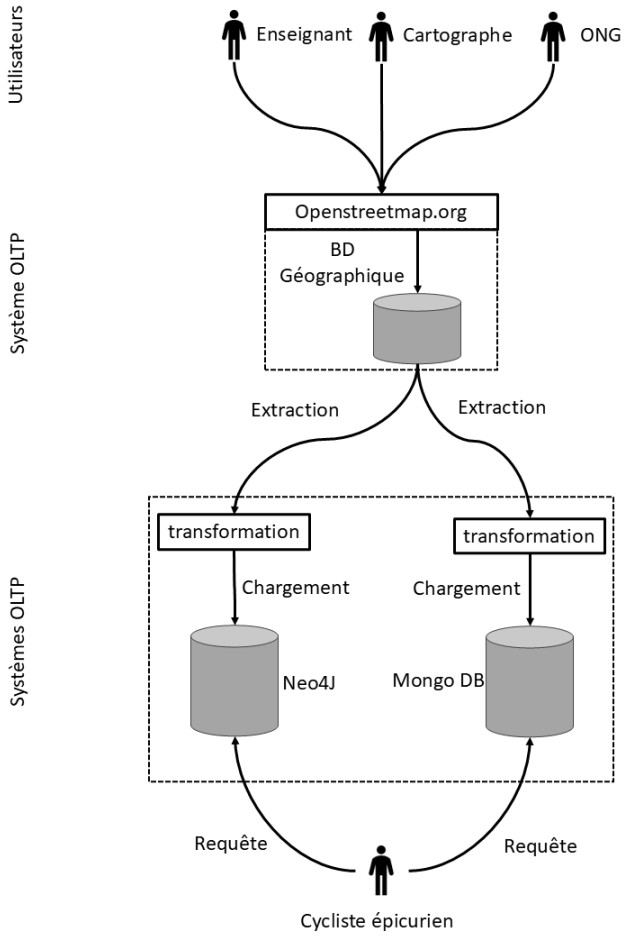
Annexe 14

Processus de transformation



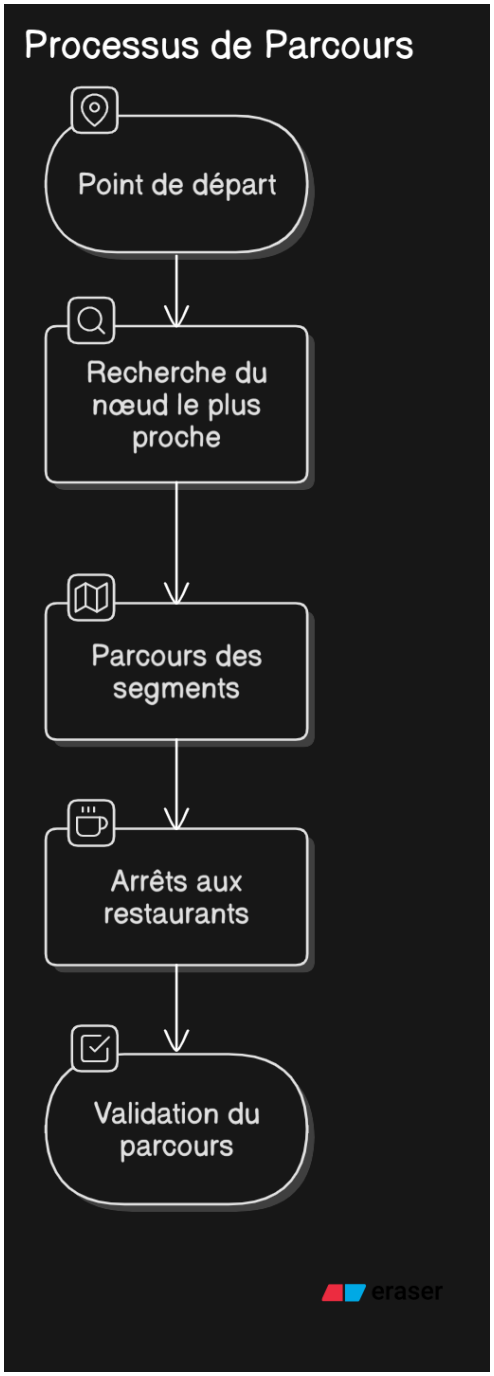
Annexe 15

Schéma du pipeline d'ETL



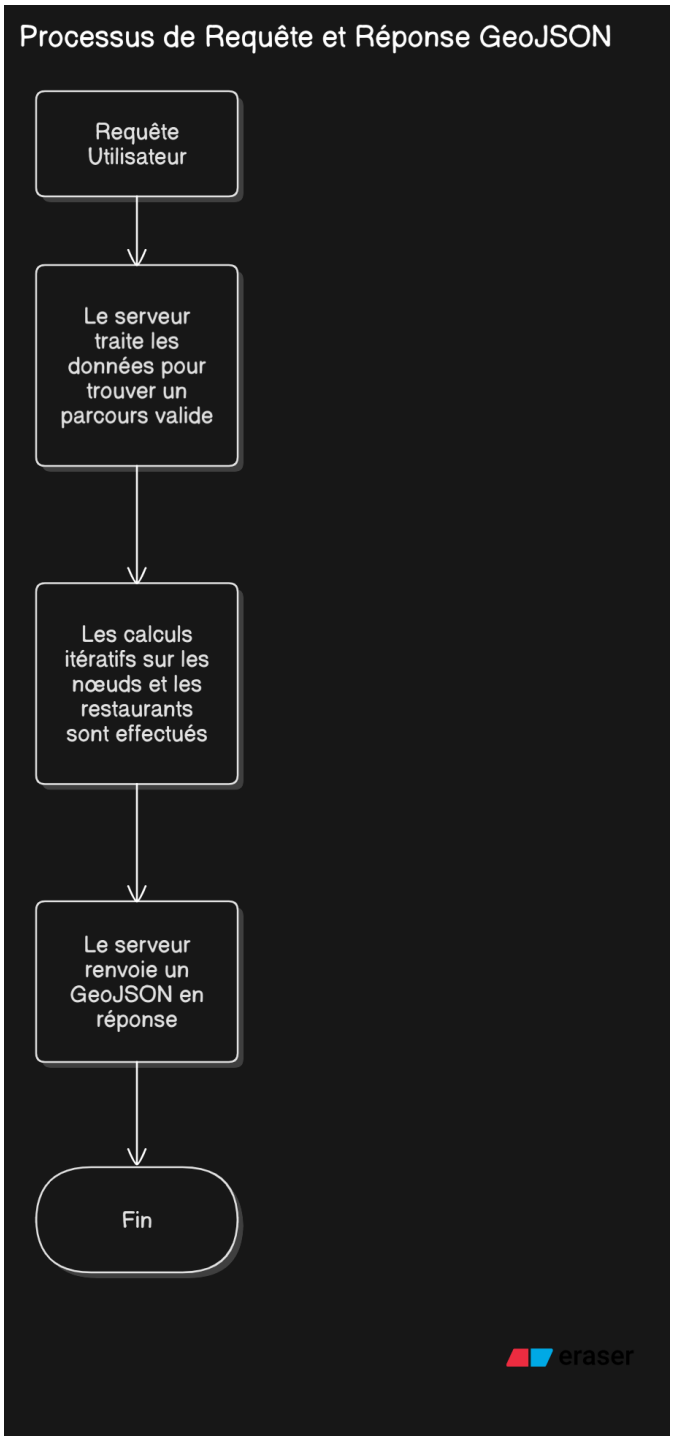
Annexe 16

Diagramme de séquence circulaire du flux de données de la procédure de création d'un parcours



Annexe 17

Diagramme de séquence circulaire du flux de données de la procédure pour obtenir un parcours



Annexe 18

Coûts estimés d’acquisition de nouvelles données

*À noter :

- Les coûts d’acquisition sont estimés selon le prix des requêtes à l’API Google Routes API, une source de qualité pour récupérer de nouvelles pistes cyclables pour n’importe quelle ville.

- Les coûts de stockage sont difficilement estimables. La littérature indique une estimation de : taille des données en Go x prix du stockage ;

Exemple dans le cas de Microsoft Azure : 10 To = 10 x 1024 x 0.002$ USD = 20.48$ USD par mois

On ne peut pas vraiment estimer ce coût pour l’instant puisque nous sommes en contexte théorique, tout dépend de l’ampleur de l’acquisition de données.

Opération	**Décomposition**	**Cas d’utilisation**	**Métrique de coûts**	**Cas de notre application**
Acquisition des données	Données externes	Acheter des données	Prix de vente	N/A, les données ne sont pas achetées dans notre cas
		Effectuer une collecte de données	Coût associé au processus de collecte	Restaurants : Gratuit grâce à OpenStreetMap Routes cyclables : 8$ USD pour 1000 requête à *Google API routes*
	Données internes	Saisir les données	Nombre d’heures * coût par heure de la saisie	N/A, les données ne sont pas saisies manuellement mais collecté

		Effectuer une collecte de données	Coût associé au processus de collecte	N/A, les données dans notre cas sont récupérées par des sources externes

Sources utilisées :

Microsoft. (n.d.). *Archive cost estimation.* Récupéré de https://learn.microsoft.com/fr-fr/azure/storage/blobs/archive-cost-estimation

Google. (n.d.). *Usage and billing.* Récupéré de https://developers.google.com/maps/documentation/routes/usage-and-billing?hl=fr

Jacky Akoka, Isabelle Comyn-Wattiau. Evaluation de la valeur des données - Modèle et méthode. Actes du 40ème congrès INFORSID, Association Inforsid, May 2022, Dijon, France. pp.163-178. ffhal-04026275f. Récupéré de https://hal.science/hal-04026275/document

HKMCI. (n.d.). *Building a traffic aware route finder with Google Maps Routes API.* Récupéré de https://www.hkmci.com/learning-articles/google-maps-platform-2/building-a-traffic-aware-route-finder-with-google-maps-routes-api/

Appleute. (n.d.). *Coûts d'exploitation d'une application.* Récupéré de https://www.appleute.de/fr/bibliotheque-de-developpeurs-dapplications/couts-exploitation-app/

Annexe 19

Coûts estimés de développement de fonctionnalités

Type de coût	Source de coût	Facteurs d'influence	Hypothèse pour notre cas
Coût direct	Équipe de développement	Complexité de la fonctionnalité, niveau des développeurs	Des développeurs juniors coûtent approximativement 40$ par heure de travail réalisé. Les coûts de développement dépendent donc de la complexité de la fonctionnalité à réaliser par ceux-ci. Le développement de la fonctionnalité en tant que telle représente environ 30 à 40% du budget alloué pour l'ajout celle-ci.
	Ajout de technologies	Complexité de la fonctionnalité, délai demandé	L'intégration de différentes technologies amènent des coûts variable. Prenons exemple de l'ajout de technologies permettant l'implantation d'une option de recherche, on estime des coûts de 300 à 500$, ou équivalent à 20 heures de travail.
	Achat de licence	Extensibilité de l'application	Dans notre cas, on peut notamment penser à l'utilisation d'une licence de Google API routes pour permettre de récupérer davantage de données de pistes cyclables pour différentes villes, estimé à environ

			8$ pour 1000 requêtes à l'API.
Coût indirect	Assurance qualité et tests	Critères d'acceptation	Les coûts de test et d'assurance qualité peuvent être estimé à environ 15-20% du budget alloué pour l'ajout d'une nouvelle fonctionnalité.
	Documentation	Norme établie par l'équipe	Il est compliqué d'estimer avec justesse les coûts de documentation. Ceux-ci dépendent du niveau de documentation souhaité pour une fonctionnalité, plus de documentation engendre plus de travail pour les développeurs et donc plus de coûts de main d'œuvre.
Coût caché	Maintenance et mises à jour	Complexité de la fonctionnalité, dette technique tolérée	Les coûts de maintenance sont réguliers et doivent être considérer continuellement dans le développement et le suivie de l'ajout d'une fonctionnalité. Ces coûts peuvent être estimés annuellement de 15 à 20% du budget alloué pour l'ajout de la fonctionnalité, pourcentage variable néanmoins selon le degré de dette technique

			toléré et l'importance de la fonctionnalité.
	Sécurité	Risques associé, niveau de confidentialité des données	Des validations de sécurité et l'intégration d'outils de surveillance peuvent être essentiel lors de l'ajout d'une fonctionnalité, dépendamment des données impliquées. Dans notre cas, avec des données géographiques publiques, la sécurité des données est négligeable. La mise en place de sécurité éventuellement pour protégée les données des utilisateurs serait toutefois à considérer dans le futur.
	Coût de fin de vie	Ressources disponibles	Planifier d'avance la dépréciation de l'efficacité et donc la valeur de la fonctionnalité dans le futur. Difficile d'estimer ce type de coût, qui sont très dépendants de la fonctionnalité impliquée et la stratégie visé par le projet en général.

Sources utilisées :

Codewave. (n.d.). *Software development costs: The true price of a project.* Récupéré de https://codewave.com/insights/software-development-costs-project/

Osman, P. (2017, 7 février). *The true cost of a feature.* Récupéré de https://medium.com/stream-zero/the-true-cost-of-a-feature-a4b80e2248a0

Understanding software development costs: A key component of project planning. (n.d.). Récupéré de https://www.linkedin.com/pulse/understanding-software-development-costs-key-component-6hnhf/

Annexe 20

Utilisation de l’intelligence artificielle

Certaines parties de ce projet ont été réalisées avec l’aide de l’assistant IA ChatGPT, en particulier pour :

1. **Script de vérification de la disponibilité de Neo4j :** Le script wait_for_neo4j() utilisé dans load.py pour s'assurer que Neo4j est prêt avant le chargement des données. Ce script attend la disponibilité de Neo4j en tentant de se connecter de manière répétée jusqu'à ce que le service soit prêt.
2. **Exécution automatique des scripts dans le Dockerfile :** La commande utilisée dans le Dockerfile pour exécuter les scripts ETL (extract.py, transform.py, load.py) a également été suggérée par ChatGPT. Cette commande garantit que le processus ETL se lance automatiquement lors du démarrage des conteneurs avec docker-compose up.
3. **Génération de diagramme UML :** Pour améliorer l’efficacité de notre création de diagramme UML, nous avons utilisé l’outil DiagramGPT. Cet outil nous a permis de générer des ébauches de diagramme basé sur notre code, que nous avons bien évidemment adapter pour qu’ils correspondent à notre travail. On a donc pu avoir plus de facilité le processus de création de diagramme UML.

Ces contributions ont permis de résoudre des problèmes techniques et d’optimiser le déploiement du projet en environnement Docker.